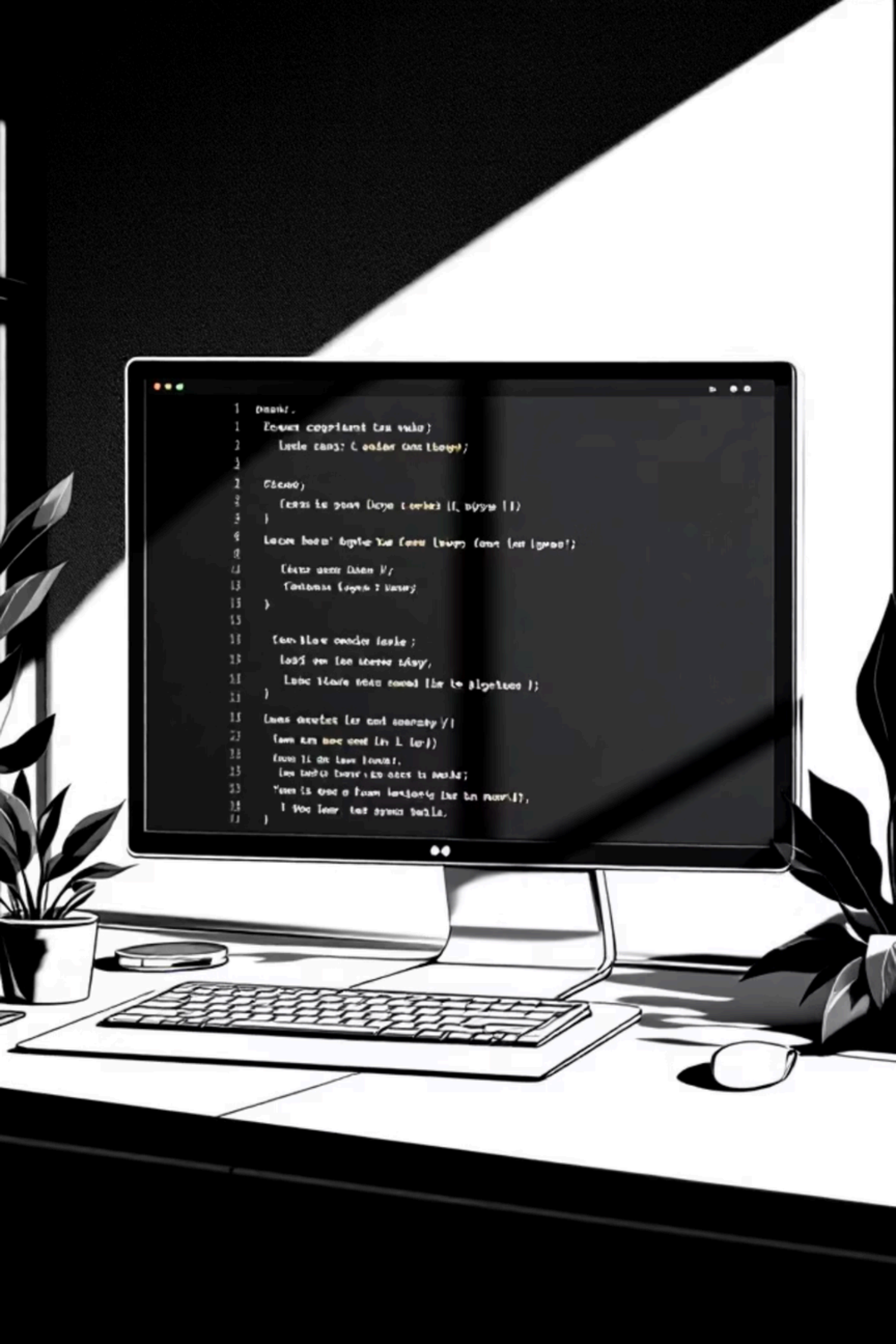




Nested While Loop in Python

Concept, Example and Applications

Name : Nitay Gupta

Class : CS1 / C15

PRN : 202501040098

Introduction to While Loops

A **while loop** repeats a block of code whilst a condition remains true. It's perfect for situations where you don't know exactly how many iterations you'll need.

A **nested while loop** places one loop inside another. The inner loop executes completely for each single iteration of the outer loop, creating powerful iteration patterns.

Basic Example

```
i = 1
while i <= 3:
    print(i)
    i += 1
```

This simple loop prints numbers 1, 2, 3 whilst the condition `i <= 3` remains true.

Understanding the Syntax

1

Outer Loop

Controls the number of iterations (rows)

2

Inner Loop

Performs the repeated task (columns)

3

Complete Cycle

Inner runs fully for each outer iteration

General Syntax Structure

```
i = 1
while i <= n:      # Outer loop condition
    j = 1
    while j <= m:  # Inner loop condition
        # Statements to execute
        # Repeated for each row
        j += 1
    i += 1
```

The outer loop variable *i* typically controls row numbers, whilst inner loop variable *j* handles column operations within each row.

Real-World Applications

Pattern Printing

Create geometric shapes, triangles, and pyramids using star patterns

Multiplication Tables

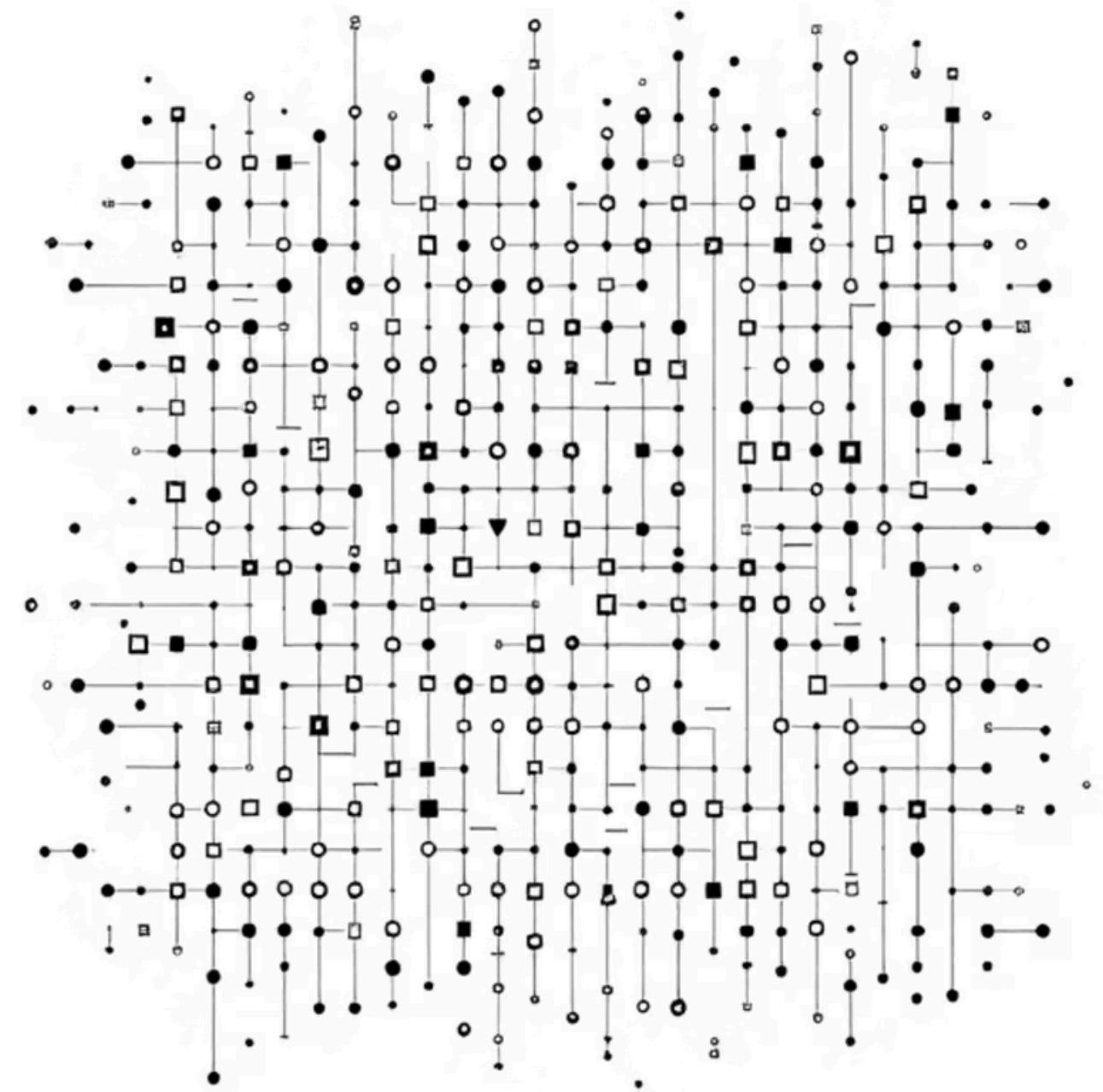
Generate complete times tables efficiently with row and column control

Grid Systems

Build matrices, tables, and coordinate-based data structures

Data Processing

Process multi-dimensional arrays and nested data collections



Practical Example: Star Pattern

Problem Statement

Print a right-angled triangle pattern using asterisks with 3 rows:

```
*  
**  
***
```

Logic Breakdown

- Outer loop controls row numbers (1 to 3)
- Inner loop prints stars based on current row
- Each row prints one more star than previous
- `print()` moves to next line after each row

Complete Code

```
i = 1  
while i <= 3:  
    j = 1  
    while j <= i:  
        print("*", end="")  
        j += 1  
    print() # New line  
    i += 1
```


Execution Flow and Output

01

Initialise Variables

Set $i = 1$ to start outer loop

03

Initialise Inner Loop

Set $j = 1$ for current row

05

Complete Row

Print new line, increment i

02

Enter Outer Loop

Check condition $i \leq 3$ is true

04

Execute Inner Loop

Print stars whilst $j \leq i$

06

Repeat or Exit

Continue until outer condition fails



Advanced Uses and Best Practices

Matrix Operations

Traverse and manipulate 2D arrays, perform matrix multiplication, or implement game boards

Complex Patterns

Generate elaborate designs like number pyramids, diamond shapes, or custom geometric patterns

Data Processing

Iterate through nested data structures, process spreadsheet-like data, or handle grid-based systems

Key Best Practices

Proper Indentation

Maintain consistent spacing to clearly show loop hierarchy

Meaningful Variables

Use descriptive names like row and col instead of generic i and j

Test Your Code

Run with small values first to verify logic before scaling up

Avoid Infinite Loops

Always ensure loop variables increment correctly to prevent endless execution